深度学习
第 6 章　PyTorch 简介

——从安装到张量，把深度学习的”乐高积木”摸透

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

学深度学习，PyTorch 就是你的工具箱。工具不熟，后面所有的模型都是空中楼阁。本章 6 道题从零打通 PyTorch 的操作基础，帮你在动手之前先在脑子里把逻辑捋清楚：

- **Q6-01（辨析填空）**：张量创建方法大梳理——`zeros`、`ones`、`rand`、`arange`、`linspace`……每种函数的参数和作用是什么？
- **Q6-02（概念理解）**：张量与 ndarray 的关系——Tensor 和 NumPy 数组怎么互转？共享内存是什么意思？什么时候会踩坑？
- **Q6-03（计算题）**：矩阵运算手动推导——给定两个张量，分别计算哈达玛积（元素级乘法）和矩阵乘法，搞清楚形状变化规律。
- **Q6-04（对比分析）**：四种索引方式横向对比——简单索引、范围索引、列表索引、布尔索引，各自的语法和适用场景是什么？
- **Q6-05（代码阅读）**：张量形状操作代码追踪——`reshape`、`squeeze`、`unsqueeze`、`transpose` 逐步追踪形状变化，搞清楚维度操作的本质。
- **Q6-06（综合应用）**：线性回归全流程——用自动微分模块手动实现梯度计算和参数更新，理解深度学习训练的底层逻辑。

1　第 6 章　PyTorch 简介

在学具体的神经网络之前，有一件事必须先搞定：**学会用 PyTorch 操作数据**。

神经网络里流动的每一份数据——图片、文字、音频——在计算机内部都是一个**张量（Tensor）**。张量就是 PyTorch 的”货币”，你必须知道怎么创建它、操作它、计算它，才能真正理解后面的模型在干什么。

- 一个数字，是 0 维张量（标量）；
- 一行数字，是 1 维张量（向量）；
- 一个表格，是 2 维张量（矩阵）；
- 一张彩色图片（高 × 宽 × 通道），是 3 维张量；

- 一批彩色图片（批量 × 高 × 宽 × 通道），是 4 维张量。

  维度越高，装的数据越复杂——但操作规律是一样的。

先建立一个总感觉：PyTorch 张量像什么？

把 PyTorch 张量想象成一个超级智能的 Excel 表格。

- 普通 Excel 是二维的（行 × 列）；张量可以是任意维度，一张图片是三维" 表格"，一批视频帧是五维" 表格"；
- Excel 只能手动计算；张量支持**自动求导**，你做完加减乘除，它能自动告诉你" 对每个参数求偏导数是多少"——这正是深度学习训练的核心引擎；
- 张量还可以住在 GPU 上，数据搬上去之后，矩阵运算速度提升成百上千倍。

本章的所有操作，都是在练习**怎么优雅地操作这个超级表格**。

Q6-01 张量创建方法大梳理

题目描述：

文档第 6.3 节介绍了多种创建张量的方式。在做题之前，先来系统梳理各类创建函数的用法。

问题：

(1) 填写下表，将各张量创建函数的关键信息补全：

函数调用示例	返回张量的内容	常见用途
`torch.zeros(3, 4)`	_____	初始化_____
`torch.ones(2, 3)`	_____	_____
`torch.arange(0, 10, 2)`	从 0 开始，步长为 2，到 10（不含）的序列：_____	_____
`torch.linspace(0, 1, 5)`	在 $[0, 1]$ 上均匀分布的____ 个数	_____
`torch.rand(3, 3)`	_____ 分布的随机数	参数_____ 初始化
`torch.randn(3, 3)`	_____ 分布的随机数	_____
`torch.full((2,3), 7)`	_____	快速创建_____ 张量

(2) `torch.arange` 和 `torch.linspace` 都能生成序列，核心区别是什么？分别在什么场景下更合适？

(3) 以下代码会输出什么形状的张量？请写出答案并解释：

```
a = torch.zeros_like(torch.rand(4, 5))
b = torch.ones_like(torch.zeros(2, 3, 4))
print(a.shape, b.shape)

```

(4) 如何用 `torch.tensor()` 从一个 Python 列表 `[[1, 2], [3, 4], [5, 6]]` 创建张量？该张量的 `shape`、`dtype` 和 `ndim` 分别是什么？

来源溯源：[文档第 6 章 6.3 张量创建]

小白必读：张量的三个最重要属性

- **shape（形状）**：张量每个维度的大小，如 `torch.Size([3, 4])` 表示 3 行 4 列。形状是理解数据结构最直观的方式，调试时第一件事就是打印 `x.shape`。
- **dtype（数据类型）**：张量中每个元素的类型。整数默认 `torch.int64`，浮点数默认 `torch.float32`。神经网络的权重必须是浮点类型，因为梯度是小数。
- **device（设备）**：张量存在 CPU（`cpu`）还是 GPU（`cuda:0`）上。两个张量必须在同一设备上才能运算，否则报错。

参考答案与详细解析

(1) 张量创建函数对比表（填写内容）：

函数调用示例	返回张量的内容	常见用途
`torch.zeros(3, 4)`	全 0 的 3×4 矩阵	初始化偏置/掩码
`torch.ones(2, 3)`	全 1 的 2×3 矩阵	构造全 1 基准张量
`torch.arange(0,10,2)`	`[0, 2, 4, 6, 8]`	生成等差序列/索引
`torch.linspace(0,1,5)`	均匀分布的 5 个数	坐标轴/插值采样
`torch.rand(3, 3)`	$[0,1)$ 均匀分布随机数	参数随机初始化
`torch.randn(3, 3)`	标准正态分布随机数	Kaiming/Xavier 初始化
`torch.full((2,3), 7)`	全为 7 的 2×3 矩阵	快速创建常数张量

(2) `arange` vs `linspace`：

- `arange(start, end, step)`：指定**步长**，元素个数由步长决定，类似 Python 的 `range`；适合需要固定间隔的整数序列或生成索引。
- `linspace(start, end, steps)`：指定**元素个数**，间隔由个数决定，端点包含在内；适合需要在区间内均匀采样的场景（如绘图坐标轴）。

(3) `zeros_like` 和 `ones_like` 的输出形状：

`zeros_like(input)` 和 `ones_like(input)` 返回与 `input` 形状相同的全 0 或全 1 张量，**忽略输入的数值，只复制形状**。

$$\texttt{a.shape} = \texttt{torch.Size([4, 5])}, \quad \texttt{b.shape} = \texttt{torch.Size([2, 3, 4])}$$

(4) 从 Python 列表创建张量：

```python
t = torch.tensor([[1, 2], [3, 4], [5, 6]])
print(t.shape)   # torch.Size([3, 2])
print(t.dtype)   # torch.int64（列表元素为整数）
print(t.ndim)    # 2（二维张量）
```

```
5
```

张量创建速查口诀

- **全 0/全 1**：`zeros` / `ones`；　**复制形状**：`zeros_like` / `ones_like`
- **固定步长序列**：`arange`；　**固定个数序列**：`linspace`
- **均匀随机**：`rand`；　**正态随机**：`randn`；　**填充常数**：`full`
- **从数据创建**：`torch.tensor(data)`

Q6-02 Tensor 与 NumPy ndarray 的相互转换

题目描述：

文档第 6.4 节介绍了张量的各种转换操作。在实际项目中，数据往往先用 NumPy 处理，再送入 PyTorch 进行深度学习计算，理解两者的转换规则至关重要。

问题：

(1) 阅读以下代码，逐行写出每个变量的类型和值：

```python
import torch
import numpy as np

a = np.array([1.0, 2.0, 3.0])
b = torch.from_numpy(a)    # 行A
a[0] = 99.0                # 行B
print(b)                   # 行C: b 的值是什么？

c = torch.tensor([4.0, 5.0, 6.0])
d = c.numpy()              # 行D
c[0] = 88.0                # 行E
print(d)                   # 行F: d 的值是什么？

```

(2) 上述代码中，修改 a 会影响 b，修改 c 也会影响 d，这是为什么？这种现象叫什么？在实际编程中会带来什么隐患，如何避免？

(3) 将 Tensor 转换为标量（Python 数值）有哪两种方式？它们有什么区别？以下代码哪一行会报错？

```python
t1 = torch.tensor([42.0])
t2 = torch.tensor([1.0, 2.0, 3.0])

val1 = t1.item()    # 方式一
val2 = t2.item()    # 方式二: 会报错吗？
val3 = float(t1)    # 方式三

```

(4) 解释 dtype 转换的两种方式，并说明深度学习中为什么通常要把整数类型的张量转换为 float32：

```python
x = torch.tensor([1, 2, 3])    # 默认 int64
y = x.float()                   # 方式一
z = x.to(torch.float32)         # 方式二

```

来源溯源：[文档第 6 章 6.4 张量转换]

小白必读：为什么 Tensor 和 ndarray 要"共享内存"？

- **共享内存的好处**：数据不需要复制一份，节省内存，转换速度极快。对于大型数据集（如数百 MB 的图像数组），复制一份代价极高。
- **共享内存的风险**：修改一个，另一个跟着变。如果代码逻辑上认为"转换后两个是独立的"，就会产生难以察觉的 bug。
- **如何"断开"共享**：转换时用 `.copy()` 或 `torch.tensor()`（注意不是 `from_numpy`）创建副本，两者就不再共享内存了。
- **GPU 上不能 `.numpy()`**：`.numpy()` 要求张量必须在 CPU 上。GPU 张量需要先 `.cpu()` 再 `.numpy()`，或用 `.cpu().detach().numpy()`（如果有梯度追踪的话）。

生活比喻：共享内存像什么？

把 `from_numpy` 想象成"共享文档"而不是"复制文档"。

- `from_numpy(a)` = 给同一份 Google 文档创建了两个快捷方式（`a` 和 `b`），打开任何一个都是同一份文档，在一处修改，另一处立刻看到变化；
- `torch.tensor(a)` = 真正复印了一份，两份文档从此独立，互不影响；
- 工作中的隐患：你以为改的是自己的那份，结果同事那边的数据也被你悄悄改掉了——深度学习里这种 bug 极难排查。

参考答案与详细解析

(1) 逐行分析：

- **行 A**：`b = torch.from_numpy(a)`——b 是与 a **共享内存**的 Tensor，形状 `[3]`，dtype `float64`；
- **行 B**：`a[0] = 99.0`——修改了底层内存的第一个元素；
- **行 C**：由于共享内存，b 的值同步变化，输出 `tensor([99., 2., 3.], dtype=torch.float64)`；
- **行 D**：`d = c.numpy()`——d 是与 c 共享内存的 ndarray；
- **行 E**：`c[0] = 88.0`——修改 Tensor 的第一个元素；
- **行 F**：d 同步变化，输出 `[88. 5. 6.]`。

(2) 共享内存（内存共享 / Memory Sharing）：

`torch.from_numpy` 和 `tensor.numpy()` 都不复制数据，转换后的对象与原对象指向同一块内存地址。修改任一方，另一方立刻可见。

实际隐患：数据预处理流水线中，若对 ndarray 做了原地修改（如归一化），会意外改变已送入 DataLoader 的 Tensor 数据。

避免方法：

```
# 断开共享，创建独立副本
b = torch.tensor(a)          # 从 ndarray 创建独立 Tensor
d = c.detach().clone().numpy()  # 从 Tensor 创建独立 ndarray

```

(3) 标量转换：

`t2.item()` 会**报错**（`RuntimeError`），因为 `.item()` 只能用于包含**单个元素**的张量。`t2` 有 3 个元素，无法转换为单一标量。

`t1.item()` 和 `float(t1)` 均正常，返回 Python 浮点数 `42.0`。

(4) dtype 转换及原因：

两种方式等价：`x.float()` 是 `x.to(torch.float32)` 的简写。

深度学习中必须用 `float32` 的原因：反向传播计算的梯度是小数，整数类型无法表示小数，也不支持自动微分（`requires_grad` 只对浮点类型有效）。`float32` 在精度和显存占用之间取得了最佳平衡，是 GPU 运算的默认类型。

Q6-03 矩阵运算手动推导：哈达玛积 vs 矩阵乘法

题目描述：

文档第 6.5 节介绍了张量的数值计算，重点区分了三种”乘法”：逐元素乘（哈达玛积）、矩阵乘法和标量乘法。这三种乘法在深度学习中各有明确的用途，混淆会导致形状报错或计算错误。

给定两个张量：

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

问题：

(1) 计算哈达玛积 $A \odot B$（逐元素乘法，对应 `A * B` 或 `torch.mul(A, B)`），写出结果矩阵。

(2) 计算矩阵乘法 $A \times B$（对应 `torch.mm(A, B)` 或 `A @ B`），写出结果矩阵，并指出结果的形状。

(3) 下面三行代码，哪些会报错？为什么？

```
C = torch.tensor([[1, 2, 3]])   # shape: [1, 3]
D = torch.tensor([[4, 5, 6]])   # shape: [1, 3]

result1 = C * D            # 哈达玛积
result2 = torch.mm(C, D)   # 矩阵乘法
result3 = C @ D.T          # D 转置后矩阵乘法

```

(4) 什么是”节省内存”（in-place 操作）？以下两种写法有什么区别？在哪种情况下不能使用 in-place 操作？

```
x = torch.tensor([1.0, 2.0, 3.0])
y = x + 1        # 普通操作
x.add_(1)        # in-place 操作（末尾加下划线）

```

来源溯源：[文档第 6 章 6.5 张量数值计算]

小白必读：三种"乘法"傻傻分不清楚？

- **标量乘法**：`A * 2`，每个元素都乘以 2，形状不变，最简单；
- **哈达玛积**（逐元素乘法）：`A * B`，两个形状完全相同的张量，对应位置元素相乘，结果形状不变。用于注意力机制的掩码、逐元素加权等；
- **矩阵乘法**：`A @ B` 或 `torch.mm(A, B)`，遵循线性代数规则——第一个矩阵的列数必须等于第二个矩阵的行数，结果形状为 $[m,n]\times[n,k]=[m,k]$。全连接层的核心计算就是矩阵乘法。
- **记忆口诀**：`*` 对元素，`@` 对矩阵。

参考答案与详细解析

(1) 哈达玛积 $A\odot B$：

对应位置元素相乘，形状保持 2×2：

$$A\odot B=\begin{pmatrix}1\times 5 & 2\times 6\\ 3\times 7 & 4\times 8\end{pmatrix}=\begin{pmatrix}5 & 12\\ 21 & 32\end{pmatrix}$$

(2) 矩阵乘法 $A\times B$：

$[2,2]\times[2,2]=[2,2]$，结果形状为 2×2：

$$A\times B=\begin{pmatrix}1\times 5+2\times 7 & 1\times 6+2\times 8\\ 3\times 5+4\times 7 & 3\times 6+4\times 8\end{pmatrix}=\begin{pmatrix}19 & 22\\ 43 & 50\end{pmatrix}$$

(3) 哪些代码报错：

- `result1 = C * D`：**不报错**。`C` 和 `D` 形状均为 `[1,3]`，逐元素乘法可以执行，结果为 `[[4, 10, 18]]`；
- `result2 = torch.mm(C, D)`：**报错**。矩阵乘法要求第一个矩阵列数等于第二个矩阵行数。`C` 形状 `[1,3]`，`D` 形状 `[1,3]`，列数 $3\neq$ 行数 1，形状不匹配；
- `result3 = C @ D.T`：**不报错**。`D.T` 形状为 `[3,1]`，`[1,3] @ [3,1] = [1,1]`，结果为 `[[32]]`（即 $4+10+18$）。

(4) In-place 操作与内存：

`y = x + 1`：创建一个新的张量 `y`，`x` 不变，额外占用内存。

`x.add_(1)`：直接在 `x` 的内存上修改，不创建新张量，节省内存。PyTorch 中所有 in-place 操作在函数名末尾加下划线（如 `add_`、`mul_`、`fill_`）。

不能使用 in-place 的情况：当张量参与了计算图（即 `requires_grad=True`）并被后续操作引用时，不能对其使用 in-place 操作，否则会破坏反向传播所需的中间值，导致梯度计算错误，PyTorch 会抛出 `RuntimeError`。

Q6-04 四种索引方式横向对比

题目描述：

文档第 6.7 节介绍了张量的索引操作。索引是"从大张量中取出你想要的那部分数据"的核心手段。四种索引方式各有语法和适用场景。

给定张量 x（形状 [4, 5]）：

```
x = torch.arange(20).reshape(4, 5)
# tensor([[ 0,  1,  2,  3,  4],
#         [ 5,  6,  7,  8,  9],
#         [10, 11, 12, 13, 14],
#         [15, 16, 17, 18, 19]])

```

问题：

(1) **简单索引**：写出以下操作的结果：

a. x[2]：_____

b. x[1, 3]：_____

c. x[-1, -1]：_____

(2) **范围索引（切片）**：写出以下操作的结果形状和内容：

a. x[1:3, 2:5]：形状_____，内容_____

b. x[:, 0]：形状_____，内容_____

c. x[::2, ::2]：形状_____，内容_____

(3) **列表索引**：x[[0, 2, 3]] 和 x[:, [1, 3]] 分别返回什么？形状各是多少？

(4) **布尔索引**：以下代码的作用和输出是什么？布尔索引在深度学习中最常用于什么场景？

```
mask = x > 10
result = x[mask]
print(result.shape, result)

```

来源溯源：[文档第 6 章 6.7 张量索引操作]

小白必读：索引不改变原张量，切片返回视图！

- **索引与切片的区别**：x[2] 是取一整行，结果是 1 维向量；x[2:3] 是取一个范围（虽然只有一行），结果仍是 2 维矩阵。维度会减少是索引的特点，切片保留维度。
- **切片是视图（View）**：切片不复制数据，修改切片会改变原张量。布尔索引和列表索引则会返回副本。
- **负数索引**：-1 表示最后一个，-2 表示倒数第二个，和 Python 列表一样，非常实用。
- **步长切片 ::2**：表示每隔一个取一个，相当于"隔行/列采样"，常用于降采样。

参考答案与详细解析

(1) 简单索引：

a. `x[2]`：第 3 行，结果为 `tensor([10, 11, 12, 13, 14])`，形状 `[5]`；

b. `x[1, 3]`：第 2 行第 4 列，结果为标量 `tensor(8)`；

c. `x[-1, -1]`：最后一行最后一列，结果为标量 `tensor(19)`。

(2) 范围索引：

a. `x[1:3, 2:5]`：取第 1、2 行（不含 3）、第 2 至 4 列（不含 5）。形状 `[2, 3]`，内容：

$$\begin{pmatrix} 7 & 8 & 9 \\ 12 & 13 & 14 \end{pmatrix}$$

b. `x[:, 0]`：所有行的第 0 列。形状 `[4]`，内容 `tensor([0, 5, 10, 15])`；

c. `x[::2, ::2]`：每隔一行、一列取一个。形状 `[2, 3]`，内容：

$$\begin{pmatrix} 0 & 2 & 4 \\ 10 & 12 & 14 \end{pmatrix}$$

(3) 列表索引：

- `x[[0, 2, 3]]`：取第 0、2、3 行，形状 `[3, 5]`，内容为第 0、2、3 行组成的矩阵；
- `x[:, [1, 3]]`：取所有行的第 1、3 列，形状 `[4, 2]`，内容为第 1 列 `[1,6,11,16]` 和第 3 列 `[3,8,13,18]` 组成的矩阵。

(4) 布尔索引：

`mask = x > 10` 生成一个形状为 `[4, 5]` 的布尔张量，元素大于 10 的位置为 `True`。

`x[mask]` 返回所有 `True` 位置对应的元素，结果是一**维张量**（布尔索引总是将结果" 拍平"）：

`tensor([11, 12, 13, 14, 15, 16, 17, 18, 19])` 形状 `[9]`

深度学习中的常见用途：用布尔索引筛选满足条件的样本（如过滤掉损失超过阈值的异常样本）、在目标检测中筛选置信度大于阈值的候选框。

Q6-05 张量形状操作代码追踪

题目描述：

文档第 6.8 节介绍了张量的形状操作，包括维度交换、形状调整、维度增删。形状操作是深度学习代码中最高频的操作之一——模型输入输出形状不对、批量维度缺失，都会在这里出问题。

阅读以下代码，逐步填写各变量的形状：

```python
import torch

x = torch.arange(24)                    # 步骤1
a = x.reshape(2, 3, 4)                  # 步骤2
b = a.transpose(0, 2)                   # 步骤3
c = a.reshape(6, -1)                    # 步骤4
d = a.unsqueeze(0)                      # 步骤5
e = d.squeeze(0)                        # 步骤6
f = torch.cat([a, a], dim=0)            # 步骤7（来自6.9节）
g = torch.stack([a, a], dim=0)          # 步骤8（来自6.9节）

```

问题：

(1) 依次写出步骤 1 至步骤 8 每个变量的形状（shape）：

a. x.shape：_____

b. a.shape：_____

c. b.shape：_____

d. c.shape：_____

e. d.shape：_____

f. e.shape：_____

g. f.shape：_____

h. g.shape：_____

(2) reshape 和 view 都能调整形状，两者有什么区别？什么情况下 view 会失败？

(3) torch.cat 和 torch.stack 都能拼接张量，核心区别是什么？用一句话分别说明各自的使用场景。

(4) 在实际深度学习代码中，unsqueeze(0) 最常用于什么目的？举一个具体的场景。

来源溯源：[文档第 6 章 6.8 张量形状操作 / 6.9 张量拼接操作]

小白必读：形状操作不改变总元素个数！

- **reshape 的铁律**：调整前后总元素数必须相等。24 个元素可以变成 `[2,3,4]`、`[6,4]`、`[24,1]` 等，但不能变成 `[5,5]`（25 个元素）。
- **`-1` 的妙用**：`reshape(6, -1)` 中 `-1` 表示"让 PyTorch 自动计算这一维的大小"。总元素 24，第一维是 6，第二维自动算出 $24/6 = 4$，所以结果是 `[6,4]`。
- **unsqueeze vs squeeze**：`unsqueeze(dim)` 在指定位置增加一个大小为 1 的维度；`squeeze(dim)` 删除指定位置大小为 1 的维度。如果 squeeze 指定的维度大小不是 1，则**不做任何操作**。
- **cat vs stack**：`cat` 在已有维度上拼接（维度数不变）；`stack` 新建一个维度再拼接（维度数加一）。

参考答案与详细解析

(1) 逐步形状追踪：

步骤	变量	形状	说明
1	x	`[24]`	0 到 23 的一维序列，共 24 个元素
2	a	`[2, 3, 4]`	24 个元素重塑为三维张量
3	b	`[4, 3, 2]`	交换维度 0 和维度 2，原 $[2,3,4]$ 变为 $[4,3,2]$
4	c	`[6, 4]`	$2 \times 3 = 6$，第二维自动推算为 $24/6 = 4$
5	d	`[1, 2, 3, 4]`	在最前面增加一个大小为 1 的批量维度
6	e	`[2, 3, 4]`	删除第 0 维（大小为 1），恢复原形状
7	f	`[4, 3, 4]`	沿 dim=0 拼接两个 `[2,3,4]`，第 0 维 $2 + 2 = 4$
8	g	`[2, 2, 3, 4]`	新建 dim=0，将两个 `[2,3,4]` 堆叠成 $[2,2,3,4]$

(2) reshape vs view：

两者功能相同，但 `view` 要求张量在内存中是**连续存储**的（contiguous）。如果对张量做了 `transpose`、`permute` 等操作，内存布局变得不连续，此时调用 `view` 会抛出错误。需要先调用 `.contiguous()` 使其变连续，再 `view`；或者直接用 `reshape`（会自动处理非连续情况，必要时复制数据）。

(3) cat vs stack：

- `torch.cat`：**在已有维度上合并**，不增加维度数。用于将同类数据沿批量或序列方向拼接，如将两批图像合并为一个更大的批量；
- `torch.stack`：**新建一个维度再合并**，维度数加一。用于将多个同形状张量组成新的批量维度，如将多张处理好的图像打包成一个 batch。

(4) unsqueeze(0) 的典型用途：

最常用于**为单个样本增加批量维度**，以满足模型输入格式要求。例如：

```python
# 推理时，单张图片形状为 [3, 224, 224]（通道×高×宽）
img = load_image("test.jpg")        # shape: [3, 224, 224]
img = img.unsqueeze(0)              # shape: [1, 3, 224, 224]
output = model(img)                 # 模型期望 [batch, C, H, W]

```

模型在训练时始终接收批量输入 `[batch, ...]`，推理时若直接传入单样本会形状不匹配，`unsqueeze(0)` 加上批量维度后即可正常工作。

Q6-06 综合应用：自动微分与线性回归全流程

题目描述：

文档第 6.10 节介绍了 PyTorch 的自动微分模块（`autograd`），第 6.11 节给出了线性回归案例。自动微分是深度学习的核心引擎——你只需要写出前向计算，PyTorch 就能自动算出所有参数的梯度。以下是一段手动实现线性回归的代码骨架：

```python
import torch

# 生成数据：y = 2x + 1 + 噪声
torch.manual_seed(42)
X = torch.rand(100, 1)
y = 2 * X + 1 + 0.1 * torch.randn(100, 1)

# 初始化参数
w = torch.randn(1, 1, requires_grad=True)   # 权重
b = torch.zeros(1, requires_grad=True)      # 偏置

lr = 0.1   # 学习率

for epoch in range(100):
    # 前向传播
    y_pred = X @ w + b                      # 行A

    # 计算损失（MSE）
    loss = ((y_pred - y) ** 2).mean()       # 行B

    # 反向传播
    loss.backward()                         # 行C

    # 参数更新（不计入计算图）
    with torch.no_grad():
        w -= lr * w.grad                    # 行D
        b -= lr * b.grad                    # 行E
        w.grad.zero_()                      # 行F
        b.grad.zero_()                      # 行G

```

问题：

(1) **`requires_grad=True` 的作用**：为什么只有 `w` 和 `b` 设置了 `requires_grad=True`，而 `X` 和 `y` 没有？

(2) **计算图（Computation Graph）**：执行行 A 和行 B 之后，PyTorch 内部构建了一张计算图。请用文字描述这张图的大致结构：从 `w`、`b`、`X` 出发，经过哪些操作，最终得到 `loss`？

(3) **`loss.backward()` 的作用**（行 C）：执行这行后，`w.grad` 和 `b.grad` 里存的是什么？数学上对应什么量？

(4) **为什么参数更新要放在 `with torch.no_grad()` 里**（行 D、E）？如果去掉这个上下文管理器，会发生什么？

(5) **为什么要手动清零梯度**（行 F、G）？如果不清零，连续训练两个 epoch 会出现什么问题？

小白必读：自动微分到底在”自动”什么？

- **核心思想**：PyTorch 在你做前向计算时，偷偷记录了所有操作（构成”计算图”）。调用 `backward()` 时，它沿着计算图反向走一遍，用链式法则自动算出每个参数的偏导数，存入 `.grad` 属性。
- **你不需要手推梯度公式**：不管网络多深、多复杂，只要前向计算写对了，反向传播完全自动。这就是为什么深度学习可以处理数百层的网络——手推梯度几乎不可能，但自动微分可以。
- **梯度的含义**：`w.grad` 告诉你”如果把 `w` 增加一点点，损失会增加多少倍”。梯度下降就是朝着梯度的反方向更新参数——损失减少最快的方向。

生活比喻：自动微分像导航软件的实时重算

- **前向传播** = 沿着当前路线从起点开到终点，记录每一个路口的转弯；
- **计算图** = 导航软件记录下来的完整行驶轨迹；
- **backward()** = 到达终点（算出 loss）后，倒回去问”如果在某个路口稍微左转一点，总耗时会变多还是变少？”——对每一个路口（参数）都问一遍；
- **梯度下降更新** = 根据回答，调整每个路口的决策，下次走得更快；
- **梯度清零** = 清空上次行程的记录，开始记录新一次行程——否则两次记录叠加，路口信息就乱了。

参考答案与详细解析

(1) `requires_grad=True` 的作用：

`requires_grad=True` 告诉 PyTorch：”这个张量是需要优化的参数，请追踪与它相关的所有计算，以便反向传播时计算梯度。”

`X`（输入数据）和 `y`（标签）是已知的固定数据，不需要优化，因此无需追踪梯度。只有模型的可学习参数 `w` 和 `b` 才需要通过梯度下降不断调整，所以只给它们设置 `requires_grad=True`。这也减少了不必要的内存占用和计算开销。

(2) 计算图的结构（文字描述）：

- 行 A：`X`（数据）与 `w`（参数）做矩阵乘法 → 加上偏置 `b` → 得到预测值 `y_pred`；
- 行 B：`y_pred` 减去真实值 `y` → 对差值平方 → 对所有元素求均值 → 得到标量 `loss`。

整张图的叶节点是 `w`、`b`（有梯度），根节点是 `loss`。反向传播从根节点出发，沿箭头方向逆向传播梯度。

(3) `loss.backward()` 的结果：

执行后，PyTorch 从 `loss` 出发沿计算图反向遍历，计算 $\frac{\partial \text{loss}}{\partial w}$ 和 $\frac{\partial \text{loss}}{\partial b}$，分别存入 `w.grad` 和 `b.grad`。这在数学上正是 MSE 损失对两个参数的偏导数，即梯度向量在参数空间上的分量。

(4) 为什么更新要在 `torch.no_grad()` 里：

参数更新 `w -= lr * w.grad` 本身是一次张量运算，如果在计算图追踪状态下执行，PyTorch 会把这次更新也记录进计算图，导致计算图不断扩大，内存泄漏，且下一次 `backward()` 时会把参数更新本身也当作一部分计算路径——这完全不是我们想要的。`torch.no_grad()` 告诉 PyTorch：”这段代码不需要追踪，纯粹的数值更新”，既节省内存又语义正确。

(5) 为什么要手动清零梯度：

PyTorch 的梯度是**累加**而非覆盖的。若不清零，第 2 个 epoch 的梯度会叠加在第 1 个 epoch 的梯度上，等效于用”两倍梯度”更新参数，参数更新方向和幅度均错误，训练无法正常收敛。

(6) 用 `torch.optim.SGD` 改写：

```python
import torch.optim as optim

# 在循环外定义优化器
optimizer = optim.SGD([w, b], lr=0.1)

for epoch in range(100):
    y_pred = X @ w + b
    loss = ((y_pred - y) ** 2).mean()

    optimizer.zero_grad()   # 替代行F、G：清零梯度
    loss.backward()         # 行C：反向传播（不变）
    optimizer.step()        # 替代行D、E：自动更新 w 和 b

```

自动微分三步节奏（深度学习的心跳）

- **第一步**：`optimizer.zero_grad()` ——清空上一轮的梯度记录
- **第二步**：`loss.backward()` ——从损失反向传播，计算所有参数的梯度
- **第三步**：`optimizer.step()` ——根据梯度更新参数
- **顺序铁律**：清零 → 反向 → 更新，三步缺一不可，顺序不能乱

现实应用场景

PyTorch 的 `autograd`——支撑起整个深度学习时代的底层引擎：

GPT、Stable Diffusion、AlphaFold……这些划时代的模型，底层都依赖自动微分来训练参数。研究人员只需要写出模型的前向计算逻辑，反向传播完全交给框架。PyTorch 的动态计算图（每次前向传播都重建计算图）让调试变得和普通 Python 代码一样直观——你可以在 `forward` 里加 `print`、加断点、写条件分支，而不需要像静态图框架那样提前声明所有计算。这也是 PyTorch 在研究界迅速超越 TensorFlow 的核心原因。

2 本章小结：把 PyTorch 的底层逻辑装进脑子里

学完这 6 道题，如果还觉得”操作很多、记不住”，试试把整章内容浓缩成这几句画面：

- **张量是一切的基础**：数字、向量、矩阵、图片、批量……在 PyTorch 里统统是张量，维度不同，但操作规律一样。
- **创建张量的方法分两类**：从数据创建（`tensor`）；按规则生成（`zeros`、`rand`、`arange` 等）。

- **Tensor 与 ndarray 共享内存**：`from_numpy` 和 `.numpy()` 不复制数据，修改一个会影响另一个——改用 `torch.tensor()` 创建独立副本。
- **三种乘法别搞混**：`*` 逐元素，`@` 矩阵乘，标量乘各自用途不同。
- **形状操作记住"总元素不变"**：`reshape` 自由变形，`unsqueeze/squeeze` 增删维度，`cat` 沿已有维度拼，`stack` 新建维度再拼。
- **自动微分的三步节奏**：清零梯度 → `backward()` → `step()`，这是深度学习训练的心跳，往后每个章节都是在这个框架上加料。

给零基础读者的一句总比喻

如果把深度学习比作造一辆汽车：

- **张量** = 钢板、零件——所有数据的物理形态；
- **张量操作（索引、切片、形状变换）** = 切割、焊接、打磨——把原材料加工成需要的形状；
- **自动微分** = 发动机——给整辆车提供动力，让训练可以自动推进；
- **线性回归案例** = 第一次发动引擎的测试：结构最简单，但发动机（autograd）是真实的，往后无论多复杂的模型，用的是同一台引擎。